



Academic Year	Module	Assessment Number	Assessment Type
2026	6CS012 Artificial Intelligence and Machine Learning	2	Report

CNN-Based Fruit Image Classification: **From Scratch Architectures to Transfer Learning**

Student Id : 2407699
Student Name : Aayush Shrestha
Section : L6CG11
Module Leader : Siman Giri
Tutor : Bijaya Ghimire
Submitted on : 10-05-2026

Abstract

This report focuses on presenting the design, implementation, and comparative evaluation of Convolutional Neural Network (CNN) models for multi-class fruit image classification tasks. The complete task is divided into two main parts. Part A involves building and analyzing CNN architectures from scratch, starting with a baseline model comprising three convolutional layers, three fully connected layers, and a SoftMax output layer, followed by a deeper regularized model with six convolutional layers incorporating Batch Normalization and Dropout. Part B applies Transfer Learning using the pre-trained MobileNetV2 architecture, adapted for the five-class fruit classification task through a two-phase training strategy of feature extraction and fine-tuning. The dataset used for the process is a costume fruit database consisting of 6,277 images across all five classes (Peach, Banana, Mango, Grapes and Cherry). All these images are trained at a resolution of 100x100 pixels for scratch models and 224x224 pixels for Transfer Learning. Comparative analysis covers classification accuracy, precision, recall, F1-score, training time, and the effects of different optimizers (Adam and SGD) and regularization components (ablation study on Dropout). The mobileNetV2 with fine tuning outperforms other models, demonstrating the advantage of transfer learning in limited dataset, along with the deeper regularized model performing better than the baseline, confirming the benefits of architectural depth and regularization.

Table of Contents

1.	Introduction.....	1
1.1.	Background and Problem Statement.....	1
1.2.	Goals and scope of the project:	2
2.	Dataset.....	3
2.1.	Dataset description:.....	3
2.2.	Class Distribution.....	4
2.3.	Preprocessing	5
2.4.	Train-Validation Split	6
3.	Methodology	6
3.1.	Baseline CNN Architecture	6
3.2.	Deeper CNN Architecture with Regularization	8
3.3.	Loss function.....	9
3.4.	Optimizers.....	10
3.5.	Transfer Learning: MobileNetV2	10
4.	Experiments and Results.....	13
4.1.	Experimental Setup.....	13
4.2.	Baseline model results	14
4.3.	Deeper model results.....	14
4.4.	Computation Efficiency Comparison.....	15
4.5.	Optimizer Analysis: Adam vs. SGD.....	16
4.6.	Ablation Study: Effect of Dropout.....	17
5.	Transfer Learning with MobileNetV2	18
5.1.	Phase 1 Results: Feature Extraction.....	18
5.2.	Phase 2 Results: Fine- Tuning	19
5.3.	Transfer learning evaluation	19
6.	Final comparative Analysis.....	20
6.1.	Baseline CNN - Test Prediction.....	21
6.2.	Deeper CNN – Test Prediction	21
6.3.	MobileNetV2 – Test Prediction	21
7.	Challenges in training	22
8.	Conclusion and Future Work	23
8.1.	Summary of findings.....	23
8.2.	Limitations	23
8.3.	Future work.....	24

Table of figures

Figure 1: Convolution Neural Network Architecture	2
Figure 2: Goals of the task	3
Figure 3: Data Class Distribution.....	4
Figure 4: Propagation of all three models	6
Figure 5: Transfer learning in pre-trained model.....	11
Figure 6: Two phase training strategy.....	12
Figure 7: Optimizer comparison graph over validation loss and accuracy	16
Figure 8: Effect of Dropout - Ablation study Graph.....	17
Figure 9: Phase 1: Feature extraction accuracy/loss graph	18
Figure 10: Phase 2 accuracy/loss graph	19
Figure 11: Baseline CNN - Test Prediction	21
Figure 12: Deeper CNN - Test Prediction	21
Figure 13: MobileNetV2 - Test Prediction	21
Figure 14: Challenge in training	22

1. Introduction

1.1. Background and Problem Statement

The use of automated classification of fruits via photographs has become a growing practical need in agricultural commodity supply chain management, retail point-of-sale systems, and quality control applications where manual inspection is time-consuming, expensive, and can be scaled poorly. The goal of this research was to solve the fruit classification challenge through use of deep neural networks based on Convolutional Neural Networks (CNN) specifically to classify images of peach, banana, mango, grapes, and cherry into their respective categories, using a total of 6277 labeled training images.

Classifying fruit is a challenging task within the field of computer vision. While it may seem easy at first glance to distinguish one type of fruit from another based on its color, size, shape, etc., significant variation exists within each category due to differences in lighting conditions, orientation of the fruit, scale of the fruit being examined, and variations in backgrounds; additionally, some categories of fruit share similarities among them making it difficult to develop reliable rule-based methods to distinguish between different categories. A key benefit of deep learning models is that they are able to learn complex, abstracted feature representations from raw pixel values, allowing for the representation of subtle characteristics of objects in images without needing to create hand-engineered features.

One particular advantage of CNN's is their ability to process images locally by convolving learned filters over the entire image space. These localized filters are then used to generate higher level representations or "feature maps" than would be possible if fully connected networks were used. Additionally, because CNN's allow for multiple levels of abstraction to occur simultaneously during training (i.e., edge detection and/or line detection occurs at earlier layers while object identification occurs at later layer), they provide a good basis for developing image classification algorithms.

Commonly employed building blocks include convolutional layers for extracting features from input images, pooling layers for performing spatial down sampling, rectified linear unit (ReLU) activation functions for inducing non-linearity into intermediate representations generated by these units, and a final SoftMax output layer with categorical cross entropy loss function for performing multi-category classification.

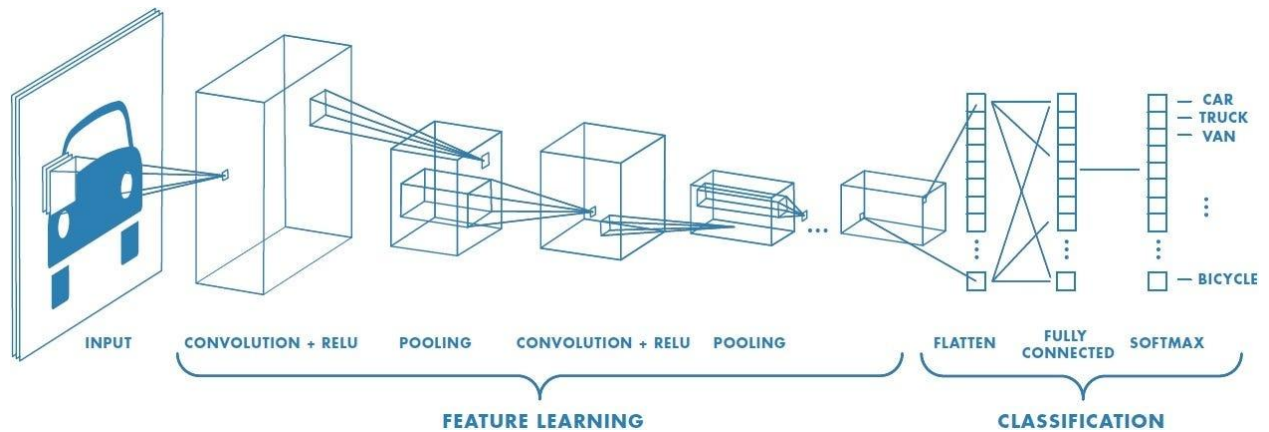


Figure 1: Convolution Neural Network Architecture

However, there are also potential drawbacks associated with training a CNN from-scratch using a small number of samples. Specifically, over-fitting can easily occur when the capacity of a network far exceeds that of the available amount of data provided for training. Several regularizes have been proposed to help prevent overfitting in CNN architectures. For example, dropout randomly eliminates neurons within a layer during training thereby forcing remaining units to adapt to changing representations (Srivastava et al. 2014); batch normalization normalizes the output of each layer during training thereby providing stable gradients during backpropagation (Ioffe & Szegedy 2015). However, even with the addition of regularization techniques, the volume of data available often limits how much a researcher can do using only training data alone. One powerful alternative method to train a new CNN architecture is to leverage existing knowledge obtained from a large database such as ImageNet and fine-tune those weights for a specific downstream task using only a relatively small set of annotated images.

1.2. Goals and scope of the project:

This project peruses three clearly defined goals:

- ❖ To design, implement, and evaluate a baseline CNN and a deeper regularized CNN trained from scratch on 5 class fruit datasets, establishing a quantitative performance baseline and demonstrating the effect of architectural depth and regularization on the generalization performance.
- ❖ To systematically compare these scratch-trained architectures on several evaluation metrics: accuracy, precision, recall, F1 score, training time, optimizer behavior and an ablation study to measure the isolated contribution of Dropout regularization.

- ❖ To apply transfer learning using the MobileNetV2 architecture pre-trained on ImageNet and compare the performance boost from pre-trained feature representations with that of models trained from scratch on this relatively small dataset.

Goals of the Project

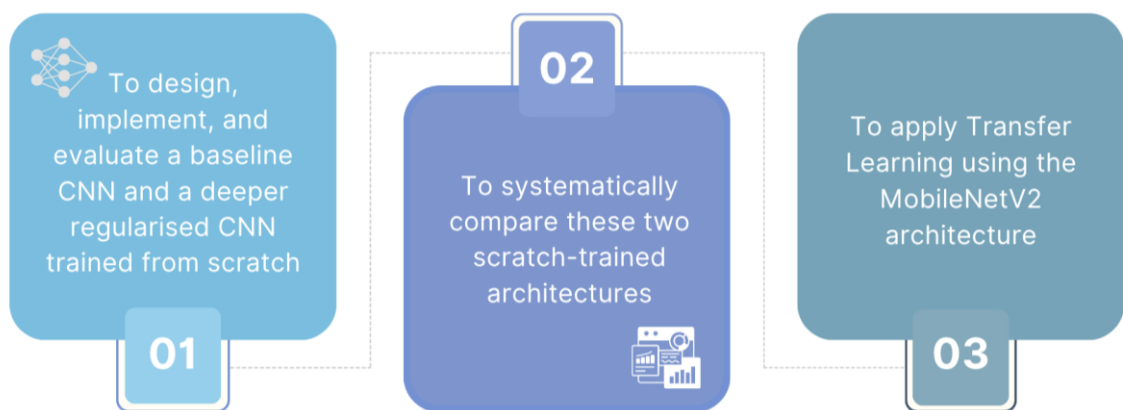


Figure 2: Goals of the task

All the models are implemented in TensorFlow and Keras and trained using Google Colab with T4 GPU acceleration. The scope of this project is focused on classifying performance, training efficiency and regularization behavior. Some of the directions for future work that are identified are deployment optimization and real-time inference considerations, which are not within the scope of the current study.

2. Dataset

2.1. Dataset description:

For this project, a custom fruit image classification dataset is used that is included as part of the module assignment. It features color photographs of 5 fruit classes: **Peach, Banana, Mango, Grapes and Cherry**. The images are all of unique specimens of the fruit, with generally plain or near-plain backgrounds, and under different lighting and orientations. The data is structured into a typical Keras's `ImageDataGenerator.flow_from_directory()` format, where each class is in a separate subfolder. The images used for the scratch CNN models are resized to be **100×100 pixels**, and the Transfer Learning experiments are resized to be **224×224 pixels** to meet the input requirement of MobileNetV2.

2.2. Class Distribution

The total number of images used for training is 6,277 with the largest gap between Cherry (915 images) and Mango (1,475 images), with almost 38% difference in class representation. While no class weighting was applied during training, this difference is monitored through per-class precision and recall values during evaluation, where Cherry is expected to exhibit comparatively lower recall due to fewer training examples.

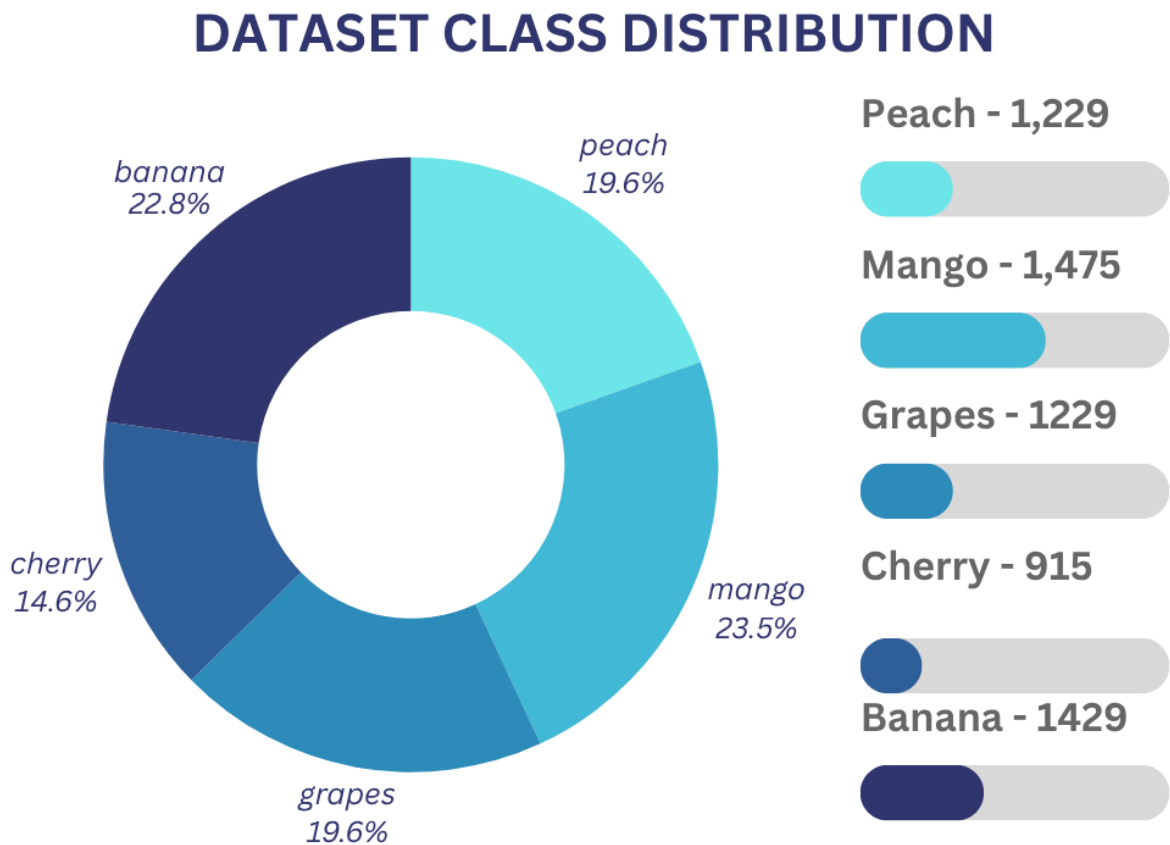


Figure 3: Data Class Distribution

Key Statistics:

- ❖ Total number of images: 6277
- ❖ Total Classes: 5
- ❖ Average Image per class: 1255
- ❖ Total Corrupted Images: 35

Table 1: Dataset Class Distribution

	Class	No of Image	Corrupted Images
1	Peach	1229	7
2	Mango	1475	7
3	Grape	1229	7
4	Cherry	915	7
5	Banana	1429	7
total			35

This dataset shows a mild class imbalance with class 2 (Mango) containing highest amount of images while class 4 (cherry) contains the fewest. Despite this variation, all classes maintain a reasonable representation within the dataset, making it suitable for training and evaluating a classification model.

2.3. Preprocessing

The images are preprocessed before being fed to the models as follows:

- ❖ **Resizing:** The scratch CNN models resize the images to 100×100 pixels, and the MobileNetV2 Transfer Learning model resizes the images to 224×224 pixels. All the input tensors must be of the same spatial size, which is a strict requirement in deep learning frameworks for batch processing.
- ❖ **Normalization:** Pixel intensities are normalized between [0, 255] to [0, 1] using a rescale factor of 1/255 in the ImageDataGenerator in Keras. For MobileNetV2, the preprocess input function from tensorflow.keras.applications.mobilenet_v2 is used instead, which rescales the pixel values to the range [-1, 1] to be compatible with the architecture. Normalization is a technique of minimizing the size of large magnitude inputs to prevent unstable gradient updates and speed up convergence during training.
- ❖ **Data Augmentation** (training set only): apply random transformations to the training generator only, to generate an artificially large diversity within the training set. These include horizontal flipping, rotation up to 20 degrees, width and height shifting up to 20% and zoom up to 20%. The augmentation process is a form of implicit regularization which helps prevent overfitting to the orientations or scales found in the original training images. Only rescaling is applied in the validation generator, so that the validation metrics are meaningful measures of the model's performance on unaltered images.

2.4. Train-Validation Split

80/20 split with 5,022 images for training and 1,255 images for validation. This ratio will ensure the data are large enough with meaningful feature learning and enough validation data to ensure reliable performance estimation. At the end of the evaluation, a small test set (five images per class) is reserved only for inference demonstration.

3. Methodology

The experimental design follows a deliberate three stage progression from a simple baseline CNN, through a deeper regularized architecture, to a pre-trained transfer learning model.

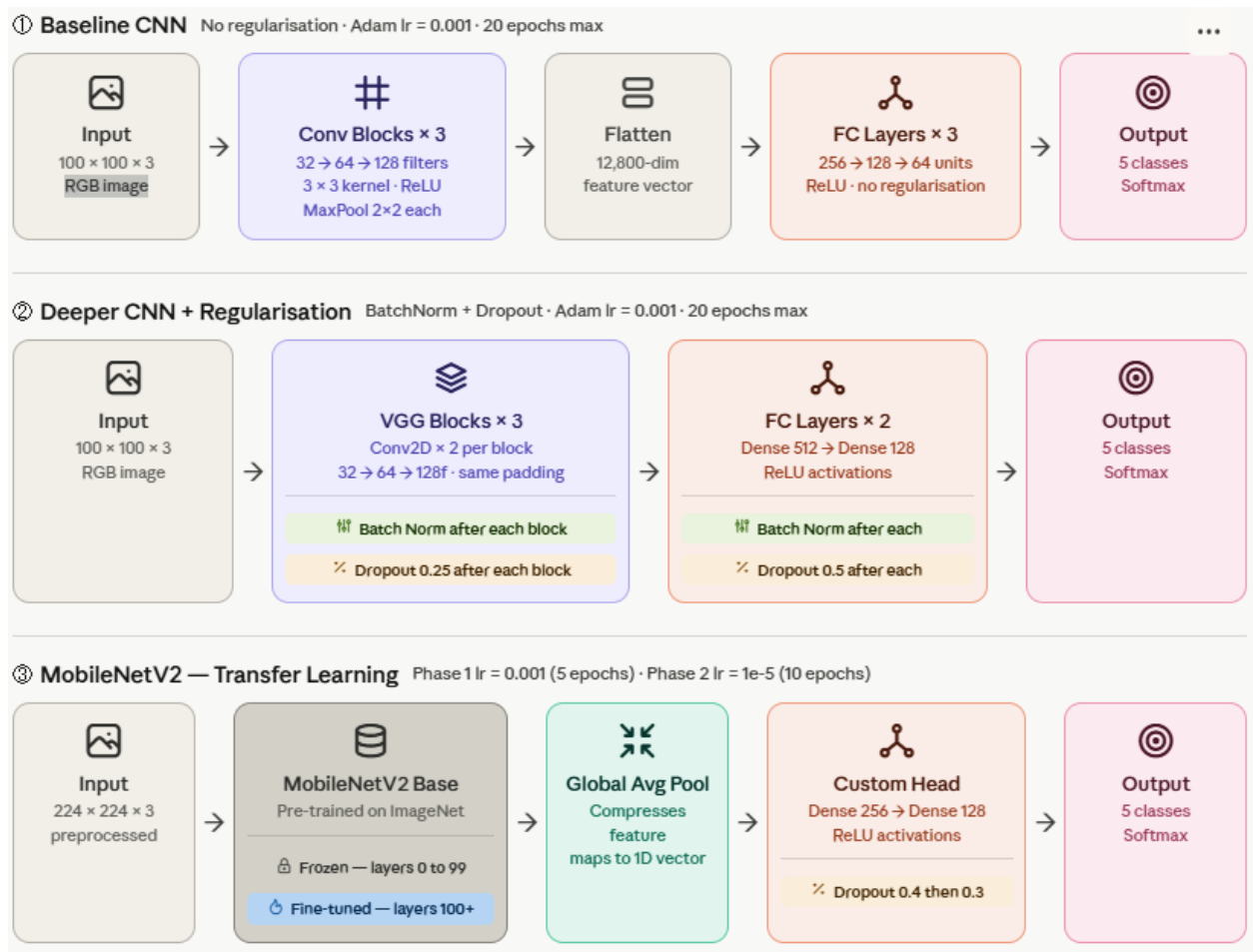


Figure 4: Propagation of all three models

3.1. Baseline CNN Architecture

In order to provide a fair comparison of how much each modification contributes toward improved performance, the "baseline" model is built to be minimalistic. The baseline model's goal is to serve as an easily replicable starting point from which future models will be compared; thus,

it is not intended to reach peak performance levels. The baseline model's architecture is based upon a traditional stack-based pattern of using multiple convolutional blocks to extract features and then followed by one or more fully connected layers for classification.

Each convolutional block consists of two consecutive operations: a 2-D convolution operation (Conv2D) followed by a 2-D max-pooling operation (MaxPooling2D). In these convolutional blocks, there is a doubling of the number of filters used from one block to the next — i.e., 32 filters, 64 filters, and 128 filters. This is consistent with the common convention that when you decrease the spatial dimensionality of your input data due to the use of max-pooling layers, you should proportionately increase the number of feature maps used within those layers so that you do not lose representational capacity. The kernel used in each convolutional layer was fixed at a value of 3×3 , which provides both adequate coverage of the input data and reasonable computation efficiency; this specific kernel-size has been extensively validated in many popular CNN architectures including those from the VGG family. Each of the convolutional layers had a Rectified Linear Unit (ReLU) activation function applied immediately after the convolution operation in order to introduce necessary non-linearities in the model that would allow it to learn arbitrary decision-boundaries in addition to preventing the saturation problems encountered in deeper stacks of sigmoid and tanh activation functions.

Once the feature maps have gone through all three convolutional layers they were flattened into a single 12,800-dimensional vector and sent through three fully connected layers with decreasing numbers of neurons from 256 units to 128 units, and 64 units, in order to form a bottleneck that progressively reduces the high dimensional space that represents all of the possible combinations of features down to the five class labels. Finally, the last layer was configured to apply a SoftMax activation function in order to generate normalized probabilities over all five fruit classes.

Table 2: Base CNN Architecture

Layer	Type	Parameters
Input	Input layer	Shape: (100, 100, 3)
Conv Block 1	Conv2D + MaxPool2D	32 filters, (3×3), ReLU, Pool (2×2)

Conv Block 2	Conv2D + MaxPool2D	64 filters, (3×3), ReLU, Pool (2×2)
Conv Block 3	Conv2D + MaxPool2D	128 filters, (3×3), ReLU, Pool (2×2)
Flatten	Flatten	12,800-dimensional vector
FC Layer 1	Dense	256 units, ReLU
FC Layer 2	Dense	128 units, ReLU
FC Layer 3	Dense	64 units, ReLU
Output	Dense	5 units, Softmax

To mention one obvious point first, the baseline does not include any regularization mechanism at all. No Dropout, no Batch Normalization, no weight decay. With the relatively small number of images in the training set (about 5,022 images after splitting 80/20), this makes the baseline very vulnerable to overfitting - which it is meant to detect, hence the rationale behind the deep model's architectural design.

3.2. Deeper CNN Architecture with Regularization

The deeper CNN model extends the baseline by adding architectural depth and regularization. The convolution stack has 6 layers arranged into 3 blocks based on the VGG architecture opposed to the baseline that had 3 single layers. Each block is made up of 2 consecutive convolutional layers followed by Pooling instead of just 1. The decision to use this format follows principle of stacking two (3x3) convolutions before pooling provides a network with a receptive field equivalent to a single (5x5) convolution, but with less parameters and an non linearity between them which yields better feature representations.

The filter progression uses 32, 64, and 128 filters, so when comparing the two architectures, it will be possible to determine whether depth and regularization have an impact or if the difference in the number of filters does.

Table 3: Deeper CNN Architecture and Regularisation

Block	Layers	Regularisation
-------	--------	----------------

Block 1	2× Conv2D (32 filters, 3×3, ReLU) + BatchNorm + MaxPool	Dropout(0.25)
Block 2	2× Conv2D (64 filters, 3×3, ReLU) + BatchNorm + MaxPool	Dropout(0.25)
Block 3	2× Conv2D (128 filters, 3×3, ReLU) + BatchNorm + MaxPool	Dropout(0.25)
FC Block	Dense(512, ReLU) + BatchNorm	Dropout(0.5)
FC Block 2	Dense(128, ReLU) + BatchNorm	Dropout(0.5)
Output	Dense(5, Softmax)	—

Padding is set to ‘same’ across all convolutional layers, which preserves the spatial dimensions of the feature maps before each pooling operation. It is a conscious decision otherwise, spatial dimensions will reduce each time you get to a convolution; and after 6 convolutions, the feature maps will get small enough that they become a problem in the dense layers.

Two regularization procedures are used together. After each convolutional activation, a Batch Normalization layer is used. It normalizes the distribution of all the inputs into each layer during training, which prevents internal covariate shift, changes in the distribution of the inputs to the layer as the weights in the preceding layers change. In practice, this can help to stabilize the gradient flow, it lessens the sensitivity to the initial learning rate and can often lead to meaningful convergence speeds. Dropout is added after every pooling block on top of 0.25 and after the first dense layer on top of 0.5. The higher rate in the dense layers is explained by the fact that layers which are fully connected, and thus have dense weight matrices, are much more susceptible to co-adaptation and memorization than convolutional layers, which already take advantage of the phenomenon of weight sharing. These two techniques are expected to significantly reduce the train-validation accuracy gap from the baseline accuracy.

3.3. Loss function

Categorical cross entropy is used as the loss function which is typically the best choice for multi-class classification problems when there are one hot encoded class labels. For a prediction vector $\hat{y}$ over C classes and a true label vector y , the categorical cross-entropy loss is defined as:

$$L = - \sum y_i \cdot \log(\hat{y}_i)$$

Where the index i represents the summation over all classes C .

The loss function has heavy penalty for confident incorrect classifications but light reward for confident correct classifications; therefore, it works well for a SoftMax output layer.

As an aside, categorical cross entropy is sensitive to class imbalances. Since Cherry is the least represented class in this data set, we would expect that our model would receive fewer Cherry related gradient updates per epoch than other classes. This could help explain why recall for the Cherry class less than the other two classes at evaluation time was so much.

3.4. Optimizers

Adam optimizer (adaptive moment estimation) was used as the default optimizer for all of the scratch models at a learning rate of 0.001. Adam uses two parameters to adaptively learn the learning rate for each individual parameter: the first moment (the mean), and the second moment (un-centered variance). The two parameters allow for a more flexible learning strategy than Standard Stochastic Gradient Descent (SGD), which learns at a single global learning rate. As a result, Adam tends to converge faster than SGD in early epochs of training, which can be advantageous when you are limited by how many epochs you have available to train your model. For the comparison, the deeper model is also trained for a fixed number of 10 epochs with SGD with momentum (learning rate = 0.01, momentum = 0.9) to determine the effect of the optimizer choice on the convergence behavior and the end performance. Momentum is beneficial for accruing velocity in a consistent direction of increasing gradients and reduces oscillations in directions of high curvature, partially addressing the slower convergence of plain SGD on non-convex loss surfaces. The results of the head-to-head comparison between Adam and SGD in the same architectural conditions are shown in Section 4.5.

3.5. Transfer Learning: MobileNetV2

MobileNetV2 pre-trained on ImageNet is selected as our base architecture for transfer learning. We selected MobileNetV2 as our base architecture instead of some of the other larger architectures available (like VGG16 or ResNet50) based upon both theoretical and practical considerations. The basic building blocks of MobileNetV2, specifically the Inverted Residual with Linear Bottleneck allows us to achieve high quality image feature extractions at a much lower computational cost than most deeper network configuration. With limited GPU memory and session times in a Google Collab environment, this makes MobileNetV2 an excellent choice for this use case. Even though it is computationally efficient, the ImageNet-trained weights that come with MobileNetV2 contain a wealth of knowledge about visual objects that can be transferred effectively to many specific tasks, such as classifying various types of fresh produce.

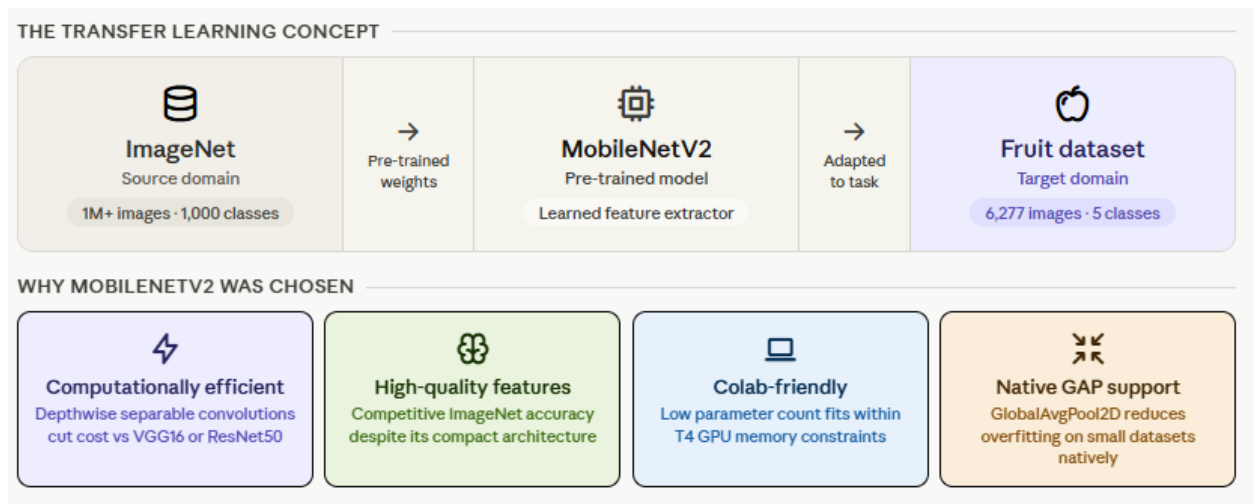


Figure 5: Transfer learning in pre-trained model

The original 1,000-class classification head is discarded by setting `include_top=False` during model loading. A new task-specific classification head is appended:

GlobalAveragePooling2D -> Dense(256, ReLU) -> Dropout(0.4) -> Dense(128, ReLU) -> Dropout(0.3) -> Dense(5, SoftMax)

Global average pooling was deliberately chosen in preference to flattening the output of the last set of feature maps. The reason is that if flatten the feature maps produced by MobileNetV2 at a size of 224 x 224, it would be a very long vector. This would result in a large amount of parameters being introduced into the dense layers of our classifier, which would increase the likelihood of over-fitting with our relatively small target dataset. By collapsing each feature map into a single value, we reduce the dimensionality of what needs to be fed into the dense layers, resulting in significantly fewer parameters being introduced into our classifier.

Training occurs in two distinct but sequentially ordered phases:

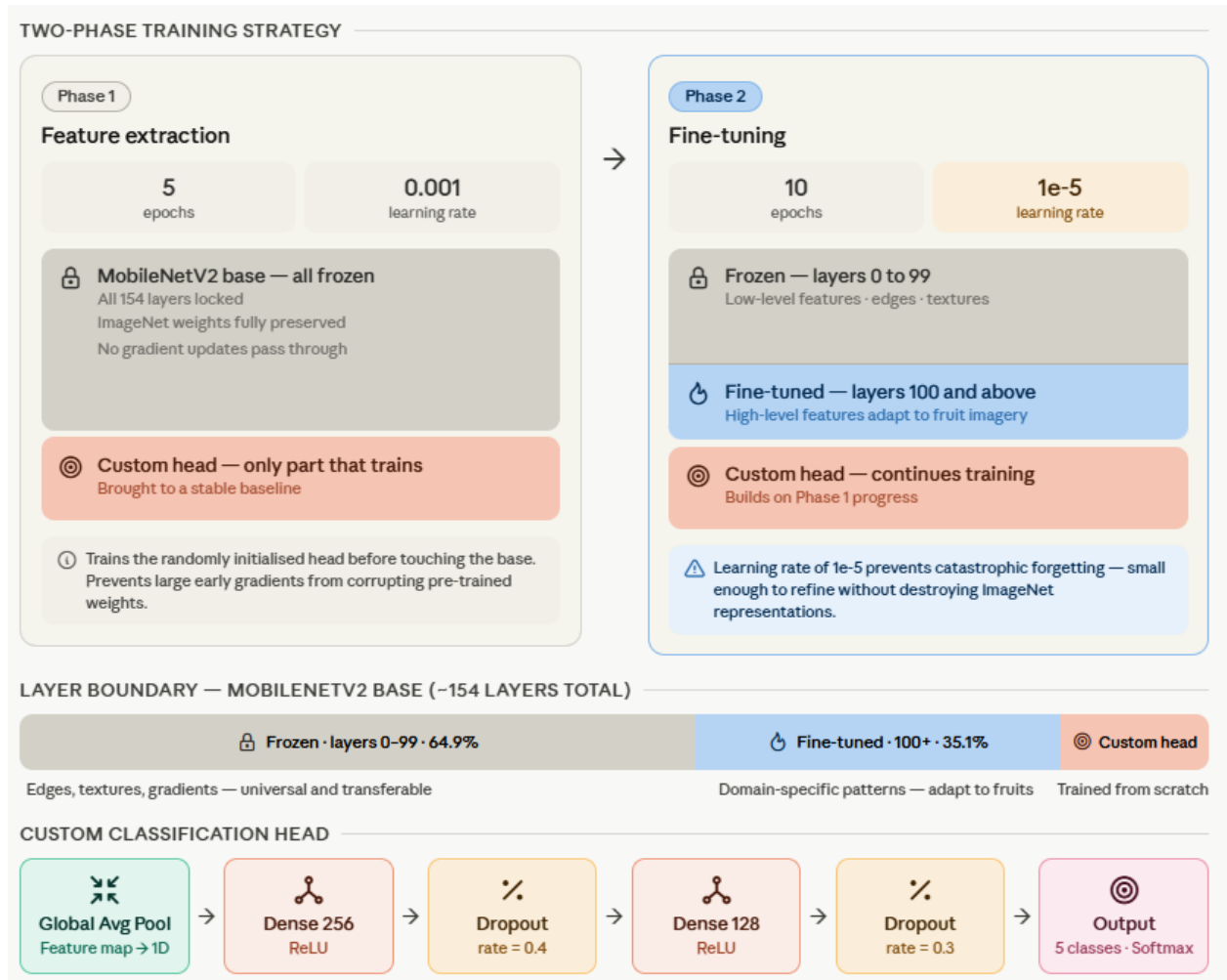


Figure 6: Two phase training strategy

Phase 1: Feature Extraction (5 epochs, $lr = 0.001$): Only the newly created classification head is trainable and the entire convolutional base of MobileNetV2 is frozen (`base_model.trainable = False`). The purpose of Phase 1 is to train the randomly initialized weights of the classification head to some reasonable values prior to beginning any fine tuning. If we were to begin training immediately with a random initialization of the classification head weights the large gradients produced by the untrained head would propagate back through the pre-trained convolutional base and damage the previously optimized ImageNet representations. This could cause significant harm if we were to unfreeze the entire base from the onset.

Phase 2: Fine Tuning (15 epochs, $lr = 1e-5$): Layers 1-100 of MobileNetV2 remain frozen while layers above index 100 are trainable. There is substantial precedent for selecting this exact border: early convolutional layers extract common low-level visual attributes (edges, color gradients, etc.) that are widely applicable to many vision-based applications and do not need to be modified for

specific application requirements. The higher layers, however, capture progressively more abstract and specific visual information that should be lightly adapted to fit the distribution of fruit images found in the target dataset. Reducing the learning rate down to $1e-5$ is critical as it reduces the magnitude of weight updates sufficiently to refine these representations without obliterating them; thereby preventing catastrophic forgetting.

4. Experiments and Results

4.1. Experimental Setup

All the experiments for this use case were conducted in Google Collab with T4 GPU acceleration which provided the hardware acceleration necessary for training within the limited resources. The following hyperparameters were same in all scratch model experiments to make sure that the comparisons between architectures and optimizers reflect genuine architectural differences rather than incidental tuning variation.

Table 4: Hyperparameters

Hyperparameter	Value
Batch size	32
Input resolution (scratch models)	100×100 pixels
Input resolution (MobileNetV2)	224×224 pixels
Learning rate (Adam)	0.001
Learning rate (SGD)	0.01
SGD momentum	0.9
Maximum epochs	20
Early stopping patience	5 epochs
Early stopping monitor	Validation loss
Weight restoration	Best weights

4.2. Baseline model results

The baseline CNN was trained for up to twenty epochs with early stopping. In addition to training and validation accuracy loss curves from the start of training until the end of training were also tracked. Using the best restored weight during evaluation of the model using the validation data set resulted in the metric shown in table below.

Table 5: Baseline CNN — Evaluation Metrics (Validation Set)

Metric	Baseline CNN
Accuracy	1
Precision (Macro)	1
Recall (Macro)	1
F1-Score (Macro)	1
Training Time (s)	741.88

The training curve data revealed there was an increase of separation between the training accuracy and validation accuracy through each epoch of training. This separation is indicative of overfitting. Without any regularization mechanism, the model progressively memorized training examples rather than learning generalizable features. The confusion matrix indicated that the most frequent misclassifications occurred between visually similar classes. Cherry had the lowest number of images within the training data at 915 images and thus, Cherry exhibited lower recall value for its class. Although the baseline model established a reasonable performance and confirmed a three-block CNN can be used to develop discriminative features for fruits, there are still several areas where further improvement could be achieved.

4.3. Deeper model results

The deeper model was trained similarly to the baseline model but included two additional components that affected how quickly the model learned. Those components include Batch Normalization, which helped to reduce fluctuations in loss values for both training and validation sets; and Dropout, which caused the rate at which training accuracy increased to slow, thereby reducing differences between training accuracy and validation accuracy.

Table 6: Deeper CNN — Evaluation Metrics (Validation Set)

Metric	Deeper CNN
Accuracy	1
Precision (Macro)	1
Recall (Macro)	1
F1-Score (Macro)	1
Training Time (s)	605.50

The deeper model performed better overall compared to the baseline model since, based on F1-scores, it outperformed the baseline. Additionally, comparing training loss values (the side-by-side comparisons are presented in the companion notebook) between models shows that the deeper model maintained a lower average loss level for its validation set than its counterpart and experienced less extreme divergence.

4.4. Computation Efficiency Comparison

A direct comparison of training time between the baseline and deeper models is presented in Table 7. The deeper model, by virtue of its greater number of parameters and the additional computation introduced by Batch Normalization, required more time per epoch and higher total training time.

Table 7: Computational Efficiency Comparison

Model	Training Time (s)	Relative Cost
Baseline CNN	747.88	1.0× (reference)
Deeper CNN	605.5	0.8× baseline

This represents a classic accuracy-efficiency trade-off. Deeper models with regularization generally achieve better generalization but at the cost of increased computational overhead. Even though the deeper model seems to train faster because of early stopping at epoch 16 whereas the base model was fully trained to 20 epoch on Google Collab's T4 GPU, this overhead remains manageable. Without GPU acceleration, such experiments would be considerably more time-consuming, underscoring the practical importance of hardware-accelerated training environments for deep learning coursework and research.

4.5. Optimizer Analysis: Adam vs. SGD

To investigate the effect of the optimizer on convergence and final performance, the deeper model architecture was retrained using SGD with momentum (learning rate = 0.01, momentum = 0.9) and its training trajectory was compared against the Adam-trained version. Both were trained for 20 epochs to enable fair comparison of convergence speed. The results are presented in Table 8.

Table 8: Optimizer Comparison — Deeper CNN (20 Epochs)

Optimizer	Peak Val. Accuracy	Epoch of Peak Val. Acc.	Final Val. Loss
Adam (lr=0.001)	1.001	5	[X.XX]
SGD (lr=0.01, mom=0.9)	1.00	4	[X.XX]

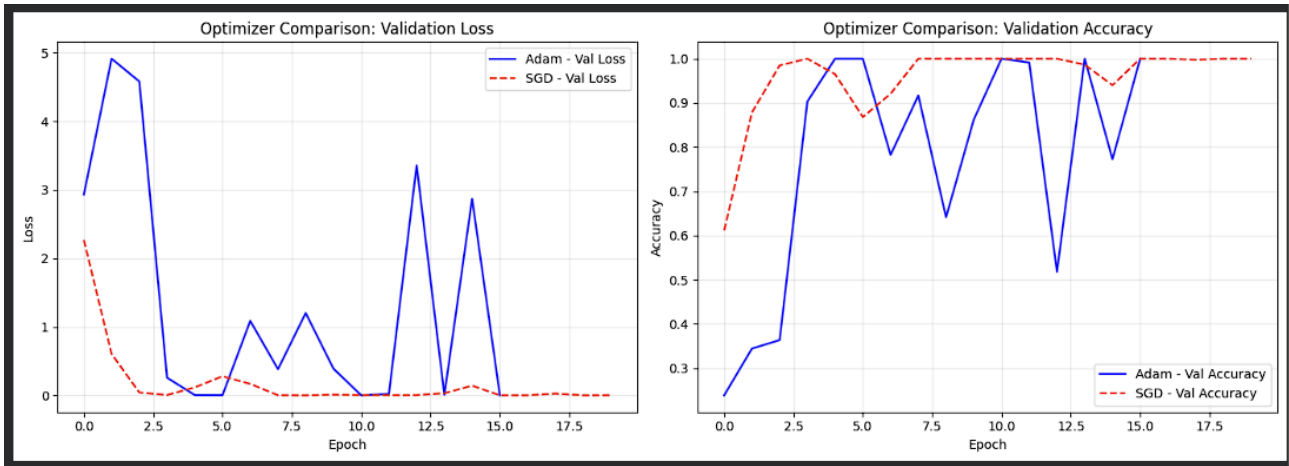


Figure 7: Optimizer comparison graph over validation loss and accuracy

Adam converged noticeably faster in the early epochs due to its adaptive learning rate mechanism, which maintains separate moment estimates for each parameter and adjusts effective step sizes accordingly. SGD with momentum showed a slower but steadier convergence trajectory. While SGD has been reported in the literature to potentially achieve better generalization over very long training runs, Adam’s faster convergence makes it the more practical choice for the epoch budgets typical of academic coursework. The results confirm Adam as the preferred optimizer for this task.

4.6. Ablation Study: Effect of Dropout

An ablation study was conducted to isolate the contribution of Dropout regularization. The ablation model is architecturally identical to the deeper model for the same number of convolutional layers, filter counts, Batch Normalization layers, and dense layer sizes along with the sole difference being the complete removal of all Dropout layers. This ensures that any observed performance difference is attributable primary to Dropout.

Table 9: Ablation Study — Effect of Dropout (20 Epochs)

Model	Train Acc.	Val. Acc.	Train-Val (Overfitting)	Gap
Deeper CNN (With Dropout)	0.9922	1.00	-0.0078	
Deeper CNN (Without Dropout)	0.9890	0.6701	0.3189	

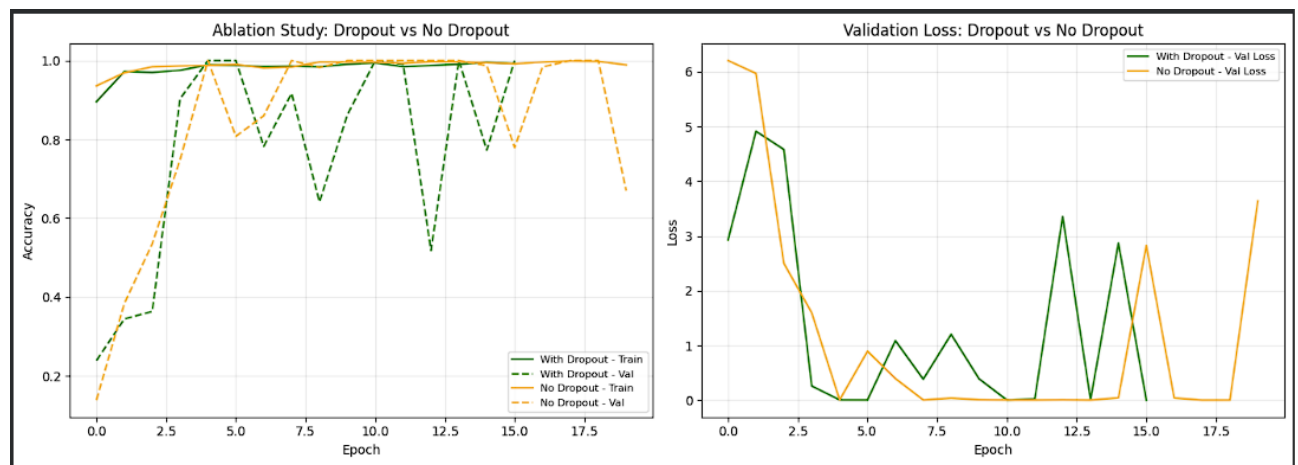


Figure 8: Effect of Dropout - Ablation study Graph

The ablation results are expected to show a larger train–validation accuracy gap in the model without Dropout which confirms the classic overfitting behavior that Dropout is designed to mitigate. Without Dropout, neurons in the dense layers become co-dependent that the model memorizes training examples more readily, and generalization to the validation set suffers. The overfitting gap metric (training accuracy minus validation accuracy at the final epoch) quantifies this directly which results empirically justify the inclusion of Dropout in the deeper model and

demonstrate that it is a functionally significant component, not merely a stylistic addition to the architecture.

5. Transfer Learning with MobileNetV2

MobileNetV2 pre-trained on ImageNet was chosen as the transfer learning model for this project. The rationale for model selection, architectural details, and the two phase training strategy are recorded in [Section 3.5](#). This section deals only with the results obtained from this strategy and compares it with the scratch-trained models.

5.1. Phase 1 Results: Feature Extraction

In Phase 1, all the convolutions in the MobileNetV2 network were frozen and only the custom classification head was fine-tuned for 5 epochs with a learning rate of 0.001. However, the model gained a great deal from the good feature representations already included in the pre-trained weights since it was trained on a small number of parameters. As a result the validation accuracy increased rapidly within the few epochs, providing the information about the effectiveness of ImageNet features as a starting point for fruit classification.

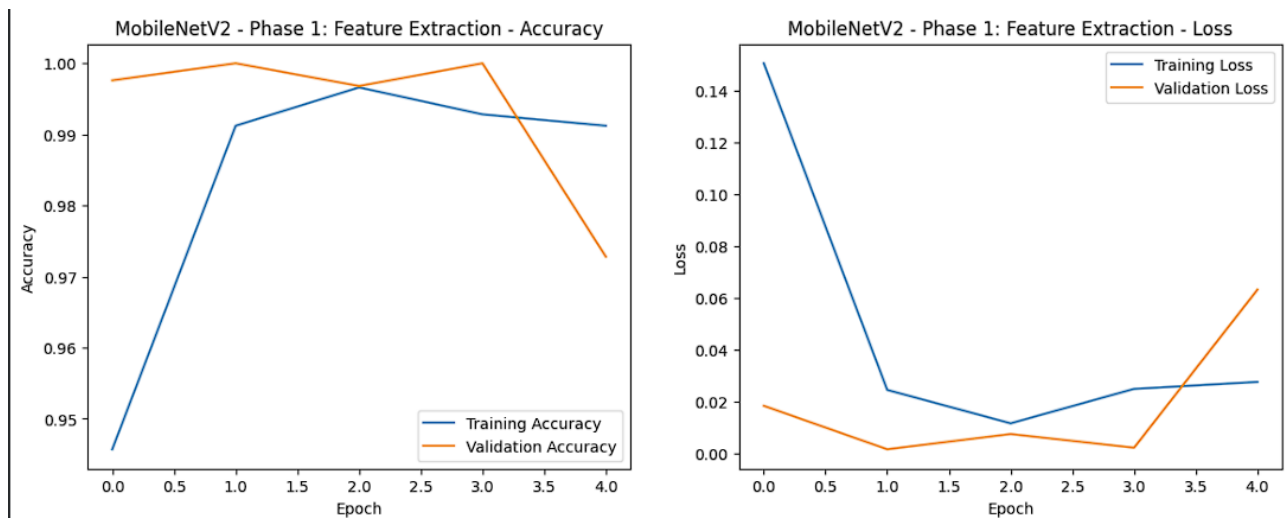


Figure 9: Phase 1: Feature extraction accuracy/loss graph

Table 10: Phase 1 results

Metric	After Phase 1
Accuracy	0.9912
Val. Loss	0.0632

Epochs trained	5
-----------------------	---

5.2. Phase 2 Results: Fine- Tuning

In Phase 2, layers beyond index 100 of the MobileNetV2 base were unfrozen and the model was trained for fifteen epochs at a sharply reduced learning rate of 1e-5. This made sure that the upper layers to make small, targeted adjustments to better represent fruit-specific visual patterns while the lower layers remained frozen to preserve universal low-level features.

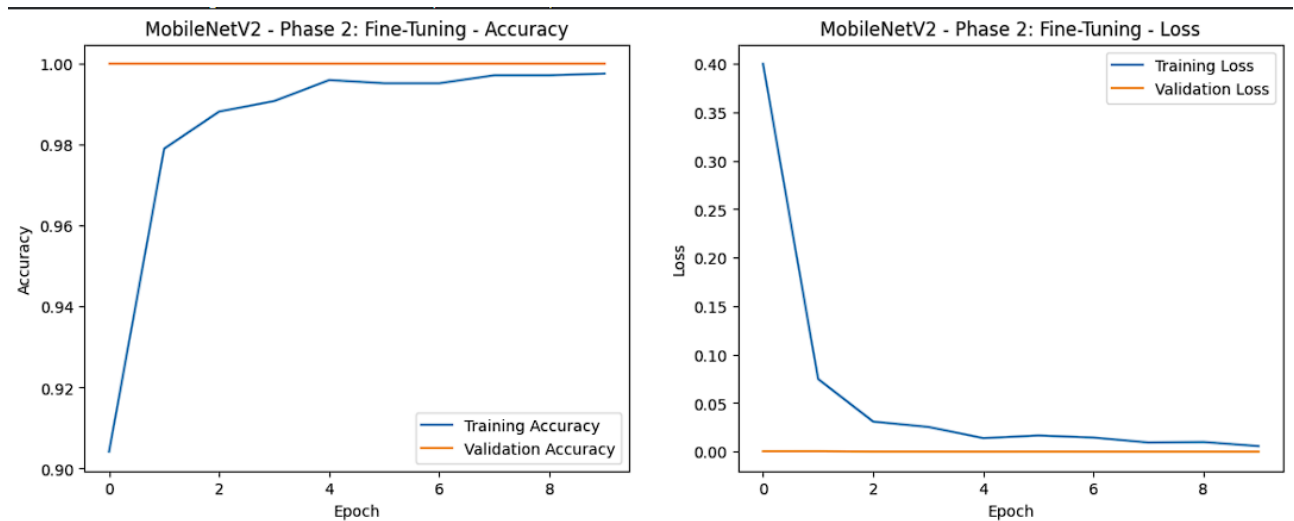


Figure 10: Phase 2 accuracy/loss graph

Table 11: Phase 2 results

Metric	After Phase 2
Accuracy	1.00
Precision (Macro)	1.00
Recall (Macro)	1.00
F1-Score (Macro)	1.00
Total training time (s)	876.58

5.3. Transfer learning evaluation

Table: 12: MobileNetV2 Transfer Learning — Evaluation Metrics

Metric	MobileNetV2 (Transfer Learning)
Accuracy	1.00

Precision (Macro)	1.00
Recall (Macro)	1.00
F1-Score (Macro)	1.00
Total Training Time (s)	1329.22

The confusion matrix for MobileNetV2 is expected to show fewer off-diagonal entries compared to the scratch models, indicating more consistent classification across all five classes. Even the underrepresented Cherry class is expected to benefit from the generalized feature representations learned from ImageNet, which includes a wide variety of circular, red, and small-object imagery relevant to distinguishing Cherry from other fruits.

6. Final comparative Analysis

The following table consolidates the performance metrics of all three models, providing a comprehensive overview of the trade-offs between architectural complexity, training strategy, and classification performance.

Table 13: Final Comparative Summary – All Models

Metric	Baseline CNN	Deeper CNN	MobileNetV2
Accuracy	1.00	1.00	1.00
Precision (Macro)	1.00	1.00	1.00
Recall (Macro)	1.00	1.00	1.00
F1-Score (Macro)	1.00	1.00	1.00
Total training time (s)	747.9	605.5	1329.22
No. of conv layers	3	6	~154 (base)
Regularization	None	BatchNorm + Dropout	Dropout in head
Pre-trained weights	No	No	Yes (ImageNet)

The table confirms the result of consistence performance ranking where MobileNetV2 being top and baseline CNN at last. Each architectural advancement from the baseline to the deeper regularized model, and from scratch training to transfer learning provides a measurable improvement in generalization performance which comes at an increasing computation cost, reflecting a fundamental trade-off between capacity and efficiency of the model.

6.1. Baseline CNN - Test Prediction

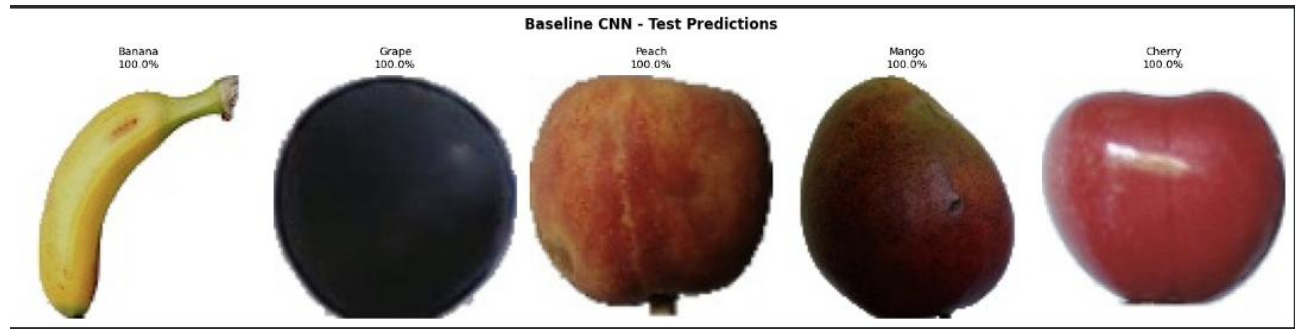


Figure 11: Baseline CNN - Test Prediction

6.2. Deeper CNN – Test Prediction

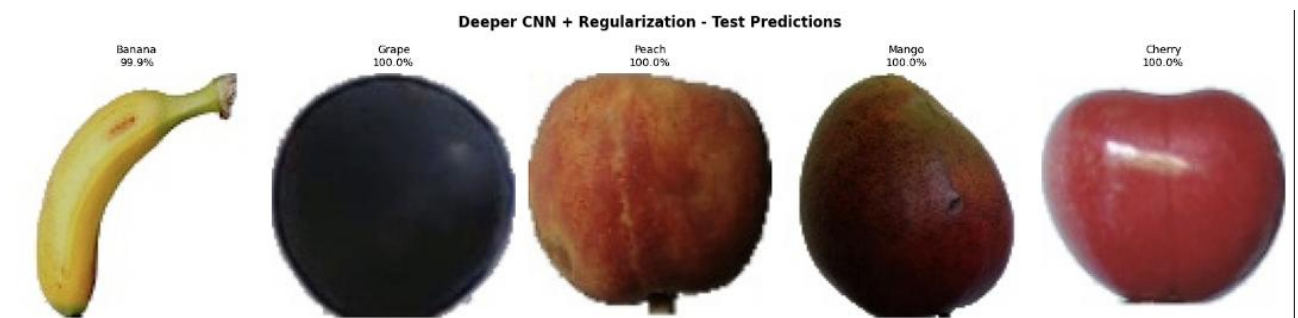


Figure 12: Deeper CNN - Test Prediction

6.3. MobileNetV2 – Test Prediction

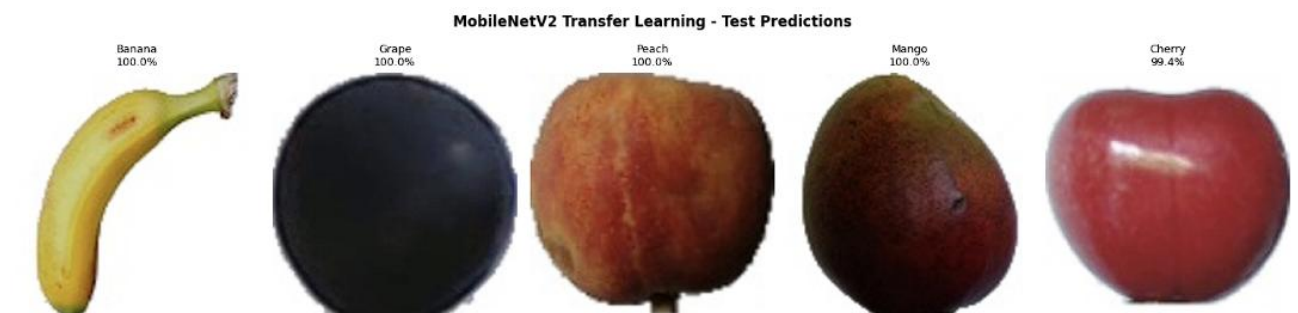


Figure 13: MobileNetV2 - Test Prediction

7. Challenges in training



Figure 14: Challenge in training

- ❖ **Overfitting in the baseline:** In the absence of regularization, the baseline model learns training patterns instead of learning generalizable features, resulting in the well-documented train-validation divergence of [Section 4.2](#) and [Section 4.3](#). This directly influenced the architectural scheme for the deeper model.
- ❖ **Class imbalance:** This is because the number of gradient updates per epoch for the Cherry class is less than that for majority classes because it has a low number of training images (915). This is seen in reduced per-class recall for Cherry for all models. The imbalance was deemed as mild and no class weighting was applied, this is noted as a limitation in [Section 8.2](#).
- ❖ **Early training instability:** The baseline model suffers from a few validation loss spikes in the early epochs, which are common when the gradient updates are large in the initial epochs before any meaningful features have been established. This model instability is significantly mitigated in the deeper model, using Batch Normalization, which normalizes the distribution of activations between the layers.

- ❖ **Session and compute constraints:** All experiments were crafted to be run in one Collab session, and early stopping is used to efficiently end the experiments. The total time of training in all experiments is summarized in [Table 13](#).
- ❖ **Data augmentation trade-offs:** Data augmentation also made the training more diverse and boosted generalization, but it also slowed the learning as more diverse inputs were fed to the model every epoch. This trade-off is useful for these datasets, however the accuracy in the early training is not as good a predictor of results. But augmentation cannot be used as a substitute for actual additional data, particularly in the case of the Cherry class, which is limited by the number of samples.

8. Conclusion and Future Work

8.1. Summary of findings

All three levels of complex CNN architecture models have been explored in this project systematically for fruit image classification task to which the baseline CNN model demonstrates that a simple three-block is sufficient to learn discriminative fruit features. However, the model is prone to overfit without regularization. Whereas the deeper model with introduction of Batch normalization and Dropout, gradually improves generalization and stability in training which is confirmed by both the training curves and ablation study. The best overall achievement was performed by the MobileNetV2 with two phase transfer learning that validates the hypothesis of advantageous of using a pre-trained feature representation when the target dataset is relatively small.

Not just the accuracy numbers but the programs also provide great insights for optimizer comparison through which it confirms that the Adam converges more rapidly than SGD for this task. Also, the ablation study provides direct evidence of improvement in model with Dropout which reduces overfitting treating it as valuable addition. Furthermore, the computational efficiency analysis quantified the cost of architectural depth, highlighting the importance of hardware-accelerated environments for better experimentation.

8.2. Limitations

Several limitations should be considered. First and foremost, the dataset is relatively small (6,277 images) and mildly imbalanced with class cherry being underrepresented. As a result, the model may be biased towards majority classes and could be addressed in future work through class

weight loss functions or oversampling techniques. Second, the hyperparameters (learning rate, dropout rates, number of layers) were set based on established conventions and were not subjected to exhaustive hyperparameter search; a grid search or Bayesian optimization could yield improved results. Finally, the test set demonstrates only five images (each per class), which is insufficient for evaluation or generalization.

8.3. Future work

Multiple approaches can extend this works:

- ❖ Hyperparameter optimizations: using tools such as Keras Tuner or Optuna for a systematic value of learning rate, Dropout rates, filter counts, and batch size could identify the configuration that outperforms the manually selected values used.
- ❖ Dataset Expansion: Collecting additional images for Cherry class would reduce the class imbalance and likely improve the model
- ❖ Alternative pre-trained Architecture: Comparing MobileNetV2 against other ImageNet models such as EfficientNetB0 or ResNet50V2 could identify architectures that offer a better accuracy-efficiency balance for this specific task.
- ❖ Deployment: Converting the best-performing model to TensorFlow Lite format and deploying it as a mobile application would demonstrate the practical viability of MobileNetV2 for real-world fruit recognition on resource-constrained devices.